

Machine Learning Interatomic Potentials with Keras API

J. P. Rili*

Department of Physics and Astronomy, University of British Columbia, Vancouver, BC V6T 1Z4, Canada

(Dated: April 3, 2024)

A neural network is used to train, predict, and evaluate a model to calculate the energies of 3-dimensional systems composed of Ti and O atoms. Python classes are implemented to quantify atomic interactions through symmetry functions, and to specify prediction algorithms. The hyperparameters of the model are optimised by minimising validation RMSE, which then produced a model that is accurate to within 100 eV. The model could be improved by proper testing of symmetry function calculations and addressing properties of features and targets.

I. INTRODUCTION

Density functional theory (DFT) is a widely-used method for calculating properties of atomic structures, which typically contain a significant number of atoms, such as the arrangement of crystals and sizable molecules [1]. While accurately predicting the structural behaviours, it has been observed that DFT is computationally expensive [1], and effort has been made in the past years into optimising algorithms for faster and more accurate computation [2].

Some studies have proposed that the use of machine learning methods can improve performance when calculating inter-atomic potentials [3, 4]. Behler and Parinello (2007), suggested the use of a neural-network (NN) model that predicts the energy of a set of atoms in bulk silicon. It was shown to be more accurate and has faster predicting times than DFT. In 2015, Artrith and Urban implemented the Behler-Parinello neural-network (BPNN) architecture in Fortran, predicting the total energies in TiO₂ crystals. In this paper, we implement and evaluate a similar, but smaller, NN through the *Keras* API [5] in *Python* and using the Atomic Energy network (aenet) TiO₂ dataset [6].

II. THEORY

The BPNN utilises a set of functions called the *radial* (RSF, denoted as G^1) and *angular symmetry functions* (ASF, G^2) to detail two and three-body atomic interactions respectively. For a system with n atoms, the radial symmetry function for an atom i with coordinates $\vec{R}_i$ is written as

$$G_i^1(\eta, R_s) = \sum_{j \neq i}^n \exp(-\eta(R_s - R_{ij})^2) f_c(R_{ij}) \quad (1)$$

and the angular symmetry function is

$$G_i^2(\eta, \lambda, \zeta) = 2^{1-\zeta} \sum_{j \neq i}^n \sum_{k > j}^n (1 + \lambda \cos(\theta_{ijk}))^\zeta \cdot \exp(-\eta(R_{ij}^2 + R_{ik}^2 + R_{jk}^2)) \cdot f_c(R_{ij}) f_c(R_{ik}) f_c(R_{jk}) \quad (2)$$

The arguments are free parameters, η can be different for (1) and (2), and $\lambda = \pm 1$. The parameters vary depending on the chemical composition, and therefore bonding, of the crystals. Here, the cutoff function $f_c(R)$ is introduced, where it describes how physically significant the atomic interaction is, written as

$$f_c(R) = \begin{cases} \frac{1}{2} \left(1 + \cos\left(\frac{\pi R}{R_c}\right) \right) & R < R_c \\ 0 & \text{otherwise} \end{cases}$$

with R_c as the cutoff parameter.

These sets of SFs are the inputs for the NN, which is composed of n identical subnets. For each atom i in the system, its associated ASF and RSF is “fed-forward” to the layers in the network to obtain an associated energy E_i , the contribution of that atom to the entire system. Then the total energy E is the linear sum of the contributions, i.e.:

$$E = \sum_i^n E_i$$

It is also important to take account of different possibilities of chemical bonds [6] like Ti–O, Ti–Ti, and O–O, which means that the parameters in the SFs may vary. To mitigate, different parameter values and combinations are added to the input, where instead of just a pair of ASF and RSF, it is a set of each; that is, the NN optimises which of the symmetries contribute more to the total energy. A schematic diagram for a subnet can be seen in Figure 1.

III. METHODS

A. Data Preprocessing

Before training and evaluating the model, the data undergoes extensive data processing. The energies (our tar-

* Email: jprili@student.ubc.ca

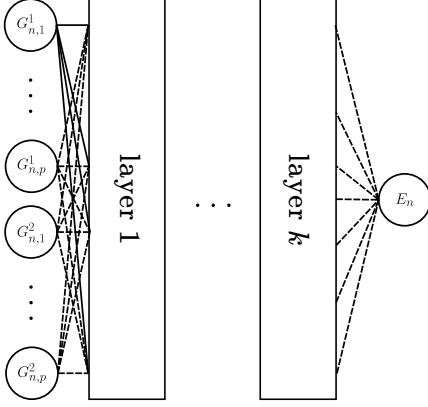


FIG. 1. The Behler-Parinello subnet for atom n . The NN uses a set of ASF and RSF, with each symmetry function denoted as $G_{i,j}^\mu$, where μ is the kind of SF, i the index of the atom in the system, and j is the index in the parameter set. There are p number of parameters for each kind of SF in this case. It is then passed forward to k layers to produce the energy contribution E_n .

gets) are extracted, and the SFs (the feature space) are computed. The aenet dataset contains 7815 *.xsf files, with each containing the total energy in eV, a set of coordinates for the unit cell of the crystal, and its primitive vectors, which dictates the vector basis of periodicity (i.e. an atom system with the origin has translational symmetry when transforming $\vec{r}_i$ into $\vec{r}_i + a\vec{v}_1 + b\vec{v}_2 + c\vec{v}_3$, with $a, b, c \in \mathbb{N}$).

The energies are extracted into a *.csv file through a Python script called `xsf_clean.py`. Then, a Python class `SymmetryCalculator` is used to calculate the SF of all atoms in each system. Specifically, the class reads the dataset and for each structure, the positions are duplicated to emulate a periodic structure using the primitive vectors and positions, then with the parameters, the G^1 s and G^2 s are calculated. The full parameter set is outlined in Tables I and II of Ref. [6], however, only 14 out of the full 70+ parameters are used due to hardware and computing time limitations. The SFs are then written to a file with the name `symXXXX.csv`, with XXXX being a four-digit index of the structure.

B. Model Implementation

As for the model itself (BPNN), it subclasses from the `keras.Model` class. The subnet takes in a list of `keras.layers` classes where it is instantiated as a `Sequential` object. When calling the object for fitting or evaluating, the second axis of the feature space with size (batch-size, n_atoms, n_symmetries) is split, and passed through the model one row at the time, where it produces an energy contribution. After iterating through all the atoms in the structure and fol-

lowing section II, we now then have a collection of contributions, where its sum is taken. All project files can be found in Ref. [7]

C. Data Analysis

The model is compiled using the mean absolute error (MAE) as loss and the root mean squared error (RMSE) as the metric to evaluate model performance. To optimise the weights and biases, we use the adaptive moment estimation (ADAM) optimisation algorithm as it is adaptive and efficient [8]. The `GridSearch` class from *Keras-Tuner's* hyperparameter (HP) tuning library is used to find the best NN. The specific values used for each HP is shown in Table I.

HP	values
number of layers	4, 5, 6
number of nodes per layer	16, 32, 64
learning rate	5×10^{-4} , 1×10^{-4} , 5×10^{-5}
dropout	True, False

TABLE I. Different values for each hyperparameter. `GridSearch` iterates through all the combinations of the values and evaluates the model to find the best-performing combination for BPNN.

IV. RESULTS

Out of 7815 structures, only 265 symmetry files are generated due to long computing times. Tuning the

HP	values
number of layers	5
number of nodes per layer	64
learning rate	1×10^{-4}
dropout	False

TABLE II. The best HPs after running `GridSearch`. The best score for the combination is RMSE = 594.4 eV.

model by minimising validation RMSE, we find the parameters in Table II. The best HPs obtained have the RMSE = 594.4 eV. Fitting the best model with batch size 20 and 80 epochs, it is seen that the model converges by epoch 10 (Figure 2), which appears to have a validation loss of ~ 600 eV MAE at higher epochs.

Its residuals have a sample mean of $\hat{\mu} = 347.7$ eV and a standard error of $\hat{\sigma} = 2464.4$ eV. $\hat{\mu}$ is also the RMSE of the testing data. Thus, it is more motivational to rewrite is as $\text{RMSE}_{\text{test}} = 327.7$ eV. Standardising and normalising the distribution produces the residual plot figure 3. The proportion of the predictions outside $1.5\hat{\sigma}$ of the true target values (outlier fraction) is equal to $z_{\text{frac}} = 0.075$.

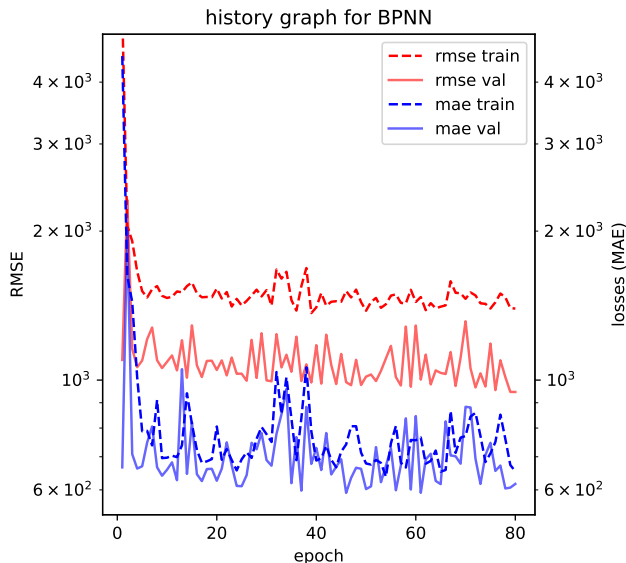


FIG. 2. History plot of the best model. The MAE drops and stabilises to around 600 eV after a few epochs, and the validation RMSE drops to 1000 eV.

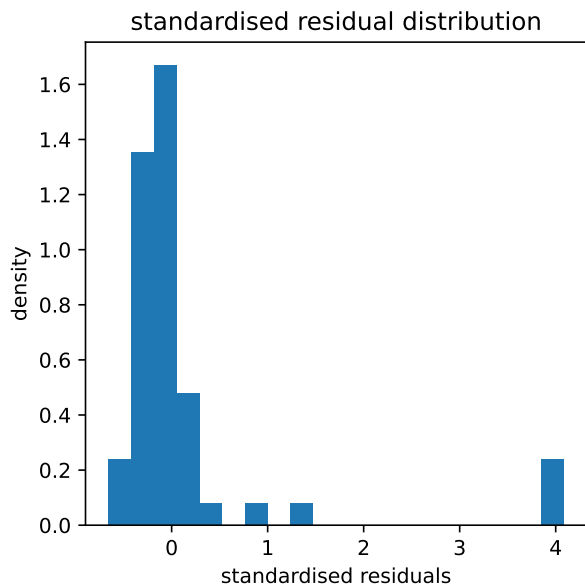


FIG. 3. Distribution of the residuals. Outliers are found for values in the $1.5\hat{\sigma}$ to $4\hat{\sigma}$ from the mean.

V. DISCUSSION & SUMMARY

Comparing $\text{RMSE}_{\text{test}}$ and the magnitude of values for the targets, it can be said that the energy calculations are accurate up to a hundred eV. The inaccuracy could be caused by several issues in the symmetry calculation or parameter fitting. Firstly, the amount of data processed is only approximately 3% of the total dataset, containing only 25 atoms per structure, whereas the dataset contains

structures with 6, 25, 47, and 95 atoms. The BPNN predicting other systems with differing unit cell size would yield great errors as uncertainties would come from extrapolation.

Another is the inaccuracy of the ASF calculation. Due to time constraints, `SymmetryCalculator` lacks proper testing so there is a possibility that the calculated RSFs and ASFs are incorrect. In the future, more optimisation should be made to the implementation such as introducing multiprocessing, rewriting the code in a compiled, lower-level language like C/C++, or integrating well-tested libraries to the implementation. Another con-

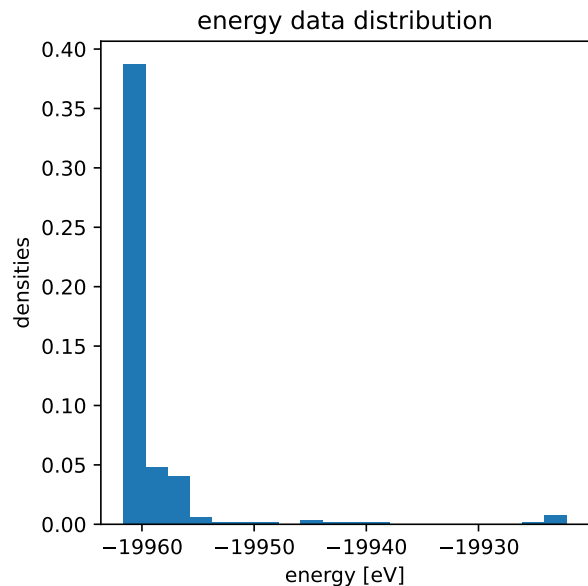


FIG. 4. Distribution of the energies in the entire dataset.

cerns the number of parameters used in the symmetry calculation. The parameters for this implementation are reduced, and this reduction makes it seem that only one chemical species is present, which will also introduce errors in prediction.

More importantly, the data imbalance that the set has affects the quality of the fitting. Standardising and normalising the distribution produces the residual plot figure 3. Inspecting the energy distribution of the dataset (see Figure 4), we see a significant amount of values that lie just under -19960 eV. This will introduce unwanted bias to the data towards its most common target value.

In summary, the BPNN is implemented in Python through the Keras API. Using radial and angular interactions of atoms, crystalline structures composed of Ti and O atoms are regressed to predict the energies of their corresponding unit cells. Tuning the hyperparameters gives predictions with accuracy up to 100 eV, which can be improved by the processing more structures and accounting for the imbalanced distribution of the energy data.

-
- [1] J.-L. Bretonnet, Basics of the density functional theory, *AIMS Materials Science* **4**, 1372 (2017).
 - [2] W. Hu and M. Chen, Editorial: Advances in density functional theory and beyond for computational chemistry, *Frontiers in Chemistry* **9**, 10.3389/fchem.2021.705762 (2021).
 - [3] J. Behler and M. Parrinello, Generalized neural-network representation of high-dimensional potential-energy surfaces, *Phys. Rev. Lett.* **98**, 146401 (2007).
 - [4] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, Physics-informed machine learning, *Nature Reviews Physics* **3**, 422 (2021).
 - [5] T. O'Malley, E. Bursztein, J. Long, F. Chollet, H. Jin, L. Invernizzi, *et al.*, Kerastuner, <https://github.com/keras-team/keras-tuner> (2019).
 - [6] N. Artrith and A. Urban, An implementation of artificial neural-network potentials for atomistic materials simulations: Performance for tio2, *Computational Materials Science* **114**, 135 (2016).
 - [7] J. P. Rili, Machine Learning Interatomic Potentials with Keras API.
 - [8] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization (2017), arXiv:1412.6980 [cs.LG].